

# Parallel Option Pricing with BSDEs Method on MapReduce

Yanxin Zhang, Bin Gong

Computer Science and Technology College, Shandong  
University,  
Jinan, Shandong, P.R. China  
zhangyanxintongxue@yahoo.com.cn  
gb@sdu.edu.cn

YingPeng, HuiLiu

Computer Science and Technology College, Shandong  
University,  
Jinan, Shandong, P.R. China  
pengy@sdu.edu.cn  
amyliuhui@sdu.edu.cn

**Abstract**—MapReduce is popular in cloud computing area. It's mainly used in Information Retrieval, Distributed Storage, DM, Machine Learning and so on. It's fit to parallel computing of great capacity for liquor data. Based on MapReduce's property, we designed a computing model for option pricing with BSDEs on it. Option pricing is one of the most important parts in financial area. To promote precision of pricing, option pricing need complex calculating with big data set. This paper shows the implementation of option pricing with BSDEs on MapReduce. It gives the detail mapper and reducer method, and displays the architecture of the model of option pricing on MapReduce. In theory, the paper analyzes its feasibility and proves that MapReduce can get great performance and nicer speedup. It can be extended in financial area.

**Keywords**- MapReduce;OptionPricing;BSDEs; Monte Carlo

## I. INTRODUCTION

From exchange-traded option in 1973, option pricing has become a very important issue in financial engineering. During the formation and development process of international finance derivatives market, the reasonable pricing of options is a major problem investors worried about. In the whole field of financial applications, option pricing is one of the most complex issues in mathematic. Option price is the only variable in options contracts that changes with market supply and demand. As the core issue of options trading, it can directly affect the level of profit and loss situation of buyers and sellers. In 1997, N. El Karoui, Shige Peng and Quenez obtained the extending of the Black-Scholes formula [1] by using Backward Stochastic Differential Equations (BSDEs). With the result, we can calculate and analyze the issue of option pricing more accurately. At present, BSDEs cover many scientific fields, especially the financial field. The theory of BSDEs can be not only used on option pricing, but also can help various kinds of investors to hedge and analyze other types of risk[2]. It can be used on incomplete markets to provide a powerful analytical and approximate calculation for all kinds of derivative securities pricing and hedging.

When the generating function is linear or other special types of nonlinear functions, with given terminal conditions and an identified generating function, the BSDEs can get explicit solution. But for most of cases, we solve the BSDEs only through numerical method. By solution principle, BSDEs can be divided into two different types. One uses nonlinear Feynman-Kac formula to transform the original

problem into similar linear parabolic PDEs. The other constructs numerical format according to the properties of BSDEs. In [3], it replaced the traditional Black-Scholes model computational method with BSDEs to solve option value. The high-precision  $\theta$ -scheme is one of the numerical methods of getting solutions, and it contains a scheme named fully discrete  $\theta$ -scheme. The method of solving the BSDEs in [3] is achieved by the fully discrete  $\theta$ -scheme through the spatial discretization and Monte Carlo simulation. This method needs large amounts of calculation and takes long time. MapReduce is one important part of Hadoop[4]. Google first proposed MapReduce in[5]. MapReduce is a simple framework. It can be run on large clusters. Customers specify a map function and a reduce function. It firstly splits a large problem into small pieces, then its runtime system can automatically run tasks parallel. MapReduce is used popular in many areas, such as Retrieval, Distributed Storage, DM, Machine Learnin, Image Processing, Biomedical filed and so on. In our paper, we firstly implement the option pricing with MapReduce as possible as we know. This is also first time that MapReduce is used in financial area.

The rest of this paper is organized as follows. Section 2 describes MapReduce basic knowledge. Section 3 introduces the mapper and reducer function for option pricing and gives the architecture of the model. Section 4 analyzes the experiment results. Section 5 shows the related work about option pricing and MapReduce. Section 6 summarizes the paper and indicates the future work.

## II. MAPREDUCE OVERVIEW

Through Google's patent 7650331 "System and method for efficient large-scale data processing" [6] mainly introduce the composition and process of MapReduce. Hadoop is an open-source project of Apache. It's mainly consisted of HDFS, MapReduce, HBase, Hive and ZooKeeper. MapReduce and HDFS(Hadoop Distributed File System) are its central members. MapReduce is a functional programming model that is well suited to parallel computation. A Map/Reduce job usually splits the input data-set into independent pieces which are processed by the map tasks in a completely parallel manner. The framework sorts the outputs of the maps, which are then input to the reduce tasks. Typically both the input and the output of the job are stored in a file-system. The framework takes care of scheduling tasks, monitoring them and re-executes the failed

tasks. The Map/Reduce framework consists of one master JobTracker and many slaves TaskTrackers(one TaskTracker per cluster-node). The master is responsible for scheduling the jobs' component tasks on the slaves, monitoring them and re-executing the failed tasks. The slaves execute the tasks as directed by the master.

MapReduce mainly contains mapper and reducer functions. Mapper maps input key/value pairs to a set of intermediate key/value pairs. Maps are the individual tasks that transform input records into intermediate records. The transformed intermediate records don't need to be the same type as the input records. A given input pair may map to zero or many output pairs. Reducer uses the output of mapper to process the problem and then output the results. MapReduce's input and output data type are as follows:

(input) <k1, v1> -> map -> <k2, v2> -> combine -> <k2, list(v2)> -> reduce -> <k3, v3> (output) .

Now MapReduce is mainly used in Web information Retrieval, Distributed Storage, Data Mining, Machine Learning, Image Processing, Biotechnology Filed and so on. This paper firstly uses MapReduce in financial area.

### III. OPTION PRICING ON MAPREDUCE

#### A. Option Pricing with BSDEs

This section introduces the option pricing definition, and then shows the BSDE mathematical model of option pricing.

##### 1) Option Pricing Overview

Option pricing is a kind of financial derivatives. An option gives the option holder the right to buy or sell a certain amount of underlying assets on a certain time in the future. According to the rights of option holders, options can be categorized into call options and put options. A call option gives the holder the right (but not the obligation) to buy a pre-specified underlying asset at a predetermined price. On the contrary, a put option gives the holder the right (but not the obligation) to sell a pre-specified underlying asset at a predetermined price. The predetermined price in the contract is called strike price. According to the execution time, options can be categorized into American-style options and European-style options. An American-style option can be exercised on any date of its maturity, while a European-style option can only be exercised at its expiration.

##### 2) BSDE Math Model for Option Pricing

We consider the numerical solutions for the backward stochastic differential equations

$$\begin{aligned} -dy_t &= f(t, y_t, z_t)dt - z_t dW_t \\ y_T &= \xi \quad t \in [0, T] \end{aligned} \quad (1)$$

where  $W_t$  is a d-dimensional Brownian motion defined on some complete, filtered probability space.

For the European call option, the terminal condition is given at the mature time T.

$$Y = \max(S_T - K, 0) \quad (2)$$

where  $S_t$  is the solution  $S_t$  of the problem at the mature time T, and K is the strike price.

Using fully discrete  $\theta$ -scheme and derivation of a series of mathematical formulas, we can get the formula for the final solution.

$$\begin{aligned} y_i^n &= E_{t_n}^{t_{n+1}}[y^{n+1}] + \Delta t_n \{ (1 - \theta_1^n) E_{t_n}^{t_{n+1}}[f(t_{n+1}, y^{n+1}, z^{n+1})] + \theta_1^n f(t_n, y_i^n, z_i^n) \} \\ 0 &= E_{t_n}^{t_{n+1}}[y^{n+1} \Delta W_{t_{n+1}}] + \Delta t_n (1 - \theta_2^n) E_{t_n}^{t_{n+1}}[f(t_{n+1}, y^{n+1}, z^{n+1}) \Delta W_{t_{n+1}}] \\ &\quad - \Delta t_n \{ (1 - \theta_2^n) E_{t_n}^{t_{n+1}}[z^{n+1}] + \theta_2^n z_i^n \} \end{aligned} \quad (3)$$

The generating function of option pricing is:

$$f(t, y, z) = -ry - \sigma^2 (\mu - r)z \quad (4)$$

where r is the return rate,  $\sigma$  is volatility,  $\mu$  is expected rate of return.

Combining the equations (3) and (4), we obtain

$$\begin{aligned} z_i^n &= \{ E_{t_n}^{t_{n+1}}[y^{n+1} \Delta W_{t_{n+1}}] + \Delta t_n (1 - \theta_2^n) E_{t_n}^{t_{n+1}}[f(t_{n+1}, y^{n+1}, z^{n+1}) \Delta W_{t_{n+1}}] \\ &\quad - \Delta t_n (1 - \theta_2^n) E_{t_n}^{t_{n+1}}[z^{n+1}] \} / (\Delta t_n \theta_2^n) \\ y_i^n &= (E_{t_n}^{t_{n+1}}[y^{n+1}] + \Delta t_n (1 - \theta_1^n) E_{t_n}^{t_{n+1}}[f(t_{n+1}, y^{n+1}, z^{n+1})] \\ &\quad - \Delta t_n \theta_1^n \sigma^{-1} (\mu - r) z_i^n) / (1 + \Delta t_n \theta_1^n r) \end{aligned} \quad (5)$$

For all the formula above, i indicates the space layer, and n indicates the time layer.

The Figure.1 shows the process of option pricing based on BSDE. After you computer the nodes on N-1 layer, you should computer the N-1 with the N layer data set. Loop the computation till the 0 time layer.

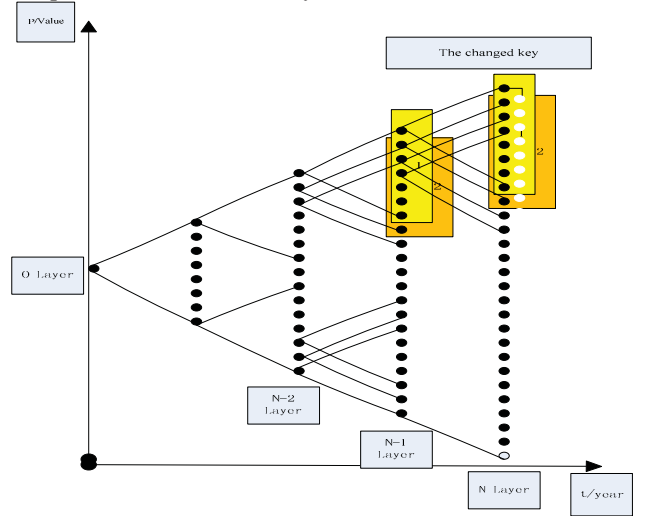


Figure1. The process of option pricing with BSDEs

#### B. MR for Option Pricing

We deploy MapReduce to parallelize option pricing with BSDEs. Because the option pricing need multi-loop to compute the 0 time price, so for every repeating, a mapper and a reducer function will be used. The mapper function offers the intermediate data for reducer. Mapper assorts the input key/value pairs to the specified key/values lists which are the reducer needs to compute one point on one time layer in the model. It's one-to-many mapping. The reducer function uses the values associated with the same key to

simulate the option price's path with Monte Carlo simulation and give out the point's option price on the time layer. After finishing the iteration, the reducer's output data is sent to mapper as input key/value pairs. Then a new iteration starts till the reducer computes the 0 time point's option price.

#### 1) key/value pairs for MR Option Pricing

The types for input key/value pairs of MR for option pricing are illustrated as follows:

Key : index→Long

Value : (x, t, y, z)→String

In key/value pairs, the index represents the point index on the current time layer in the time-space grid model. And x, t, y, z are double type. x expresses the point's space value in the time-space grid model. t represents the current time layer. y represents the option price associated to x. z represents the option risk value of x.

At first, every point on one time layer in the time-space grid model is identified by a numerical key. After being mapped in the map phase, each point will be marked more keys to fit next step computation. Then these points will be partitioned through their keys. And then these intermediate results will be sent to reducer function. In reduce phase, use the data with same key to compute the current time layer points' values.

#### 2) Map Phase

The MR for option pricing Map function, shown as Function1, is called once for every point on current time layer on the model in each steps of the loop. The key is the index of the point. And the value is the option price information associated with the point. The map function processes one key's value to more key to fit reduce phase's computing. The process is like Figure1. The points marked with the same key are for the former time layer. For example, the first point on t time layer needs the points' value on t+1 time layer to compute it's option price and risk. After processing the points, map function updates the time layer's value and submits the results using emit.

#### Function1 MR for Option Pricing Map

```
Map (key,value)
{ key is mapped to
• 1, 2, ..., key;
• key-num+1, ..., N-num+1;
• key-num+1, ..., key;
t-layer=t-layer - 1; emit (key, value);
}
```

The results generated by the map phase are stored in the local machine. Intermediate results produced by map operations on the same node will be merged by key as the input for reduce phase.

#### 3) Reduce Phase

The MR for OptionPricing reduce function is shown as Function2. It receives a key and a list of all associated values. The key is the point's id on current time layer in the time-space grid mode. The values list contains all the information for computing the option price and risk of this point associated to the key. As Function2 shows, the reduce phase contains complex computing. It should generate a set of

random numbers to simulate Brownian movement, and then use Monte Carlo simulation to simulate the option price moving path. Through this method, it uses the next time layer points' values to compute one point's price and risk on current time layer. After computing, it submits <key, value> to the HDFS. The key is the point id, and the value contains x, t, y, and z. After reduce phase completed, the map phase starts again. It means a new iteration starts. The map function will use the reduce function output as its input. The computation won't end until the loop ends.

#### Function2 MR for Option Pricing Reduce

```
reduce (key, value list)
{For (i: 1 → NE) //use MonteCarlo simulates NE times
{ Get random x, x obeys N(0,1);
Compute Brownian Motion  $W_t = x \cdot \sqrt{\Delta t}$ 
Compute  $X_j \rightarrow x_k + W_t$ ;
Use the interpolation  $I_h$  to compute the simulated Sy, Sz;
Sy +=Sy; Sz+=Sz;
}
compute the E[u] in formula (5);
compute the y, z using formula (5);
emit (key, value);
}
```

#### 4) Architecture

The architecture of the runtime system is shown as Figure2. Master assigns tasks to slaves, which execute mapper and reducer function. Mapper and reducer function are defined by users. Master splits the data set to m pieces for m map tasks. Every map task reset the data set. They output data associated with reducer function computing. Its output implements pre-shuffling [16] with mapper function arithmetic, so it avoids hot points before reducer function efficiently. The data associated with the same key is assigned to the task of reduce. A list of data as input, reducer function outputs a value for the key. Then the master collect the output of reduce tasks, and assigns the data to map tasks again. It means a new iteration begins. The iteration ends till 0 time layers.

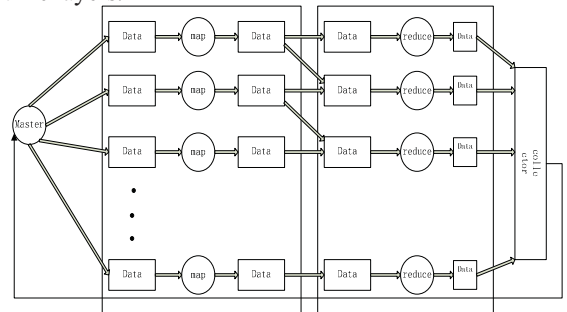


Figure2. Architecture of MR for option pricing

## IV. EXPERIMENT RESULTS

MapReduce is used popular in cloud computing area. It's a simple framework and can easily implement parallel computing. The option pricing function with BSDEs is

implemented on MapReduce for accelerating computing. And MapReduce application is expanded in financial area. MapReduce is feasible for option pricing with BSDEs.

The experiment is implemented on Linux cluster. In the cluster, every pc has Inter(R) Core(TM) i7 920 (2.67GHz) CPU and 2GB Memory. Our system is Fedora13. We implement this experiment with java (jdk1.6.0\_22) and MapReduce(hadoop-0.20.2). The application is run on Eclipse.

In our arithmetic, the mapper function prepares for reducer function. Its computation method is simple. It mainly copies the original data to a new key. So it will spend little time. After mapper function, MapReduce runtime system will deal with the output of mapper. It will spend some time. Then the reducer function works. Reducer contains complex computation, which uses Monte Carlo to simulate the path of option price. Reducer function will spend the most time of the total computing. When the number of slave nodes added, the time map tasks spent decrease slightly. Of course compared to reduce task, the time map tasks spent is much lesser. When we add slave nodes, reducing task will take less time. During the computing process, reducer function should wait mapper function, and the communication spends some time. When MapReduce launches, it takes several seconds. The speedup trend is like Figure3.

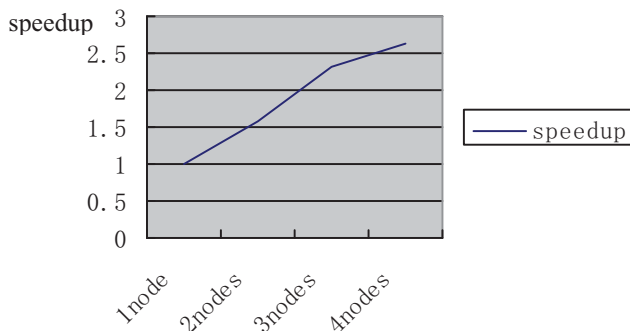


Figure3. The speedup of MR for Option Pricing

From the experiment, we know that the computing speed can be promoted and computing time can be decreased through MapReduce, although its performance needed to be improved in the future. If the computing is implemented with larger data set, it will get better speedup. If reducer method is improved, the experiment will get better performance in future work.

## V. RELATED WORK

During the financial transactions process, any delay in information processing may cause potential economic losses, so the time factor is particularly important in the financial calculations. With the development of computer technology, financial calculation efficiency has become a hotspot in the researching field. In order to improve the efficiency and accuracy of computation, a lot of people have taken great effort.

Financial derivatives pricing is one of the most complex parts in financial calculation. With the emergence of computer clusters and multi-core CPU, the technology which uses parallel computing to price financial derivatives on CPU makes great progress. In [7], Ruppa K.Thulasiram and Mmitri A.Bondarenko brought up a parallel algorithm for binary tree option pricing which realizes parallel computation by data hierarchy and MPI. In [8], by blocking the binary tree grid model, Kai Huang can compute prices of American-style Asian options. According to the properties of Monte Carlo calculation, we use Monte Carlo simulation to do a lot of work in parallel pricing derivatives. Giorgio Pauleto described a variety of parallel generation methods of pseudo-random number in [9], and calculated option prices through the Monte Carlo parallel simulation; Using the Monte Carlo simulation, Jin Suk KIM calculated the prices of financial derivatives in parallel on cluster systems, and this method accessed to a highly efficient speedup [10]. Giray Ökten and Matthew Willyard obtained a good efficiency in the parallel calculation of digital option and mortgage[11], since the use of Quasi-Monte Carlo simulation and quasi-random numbers are characteristics of this method. At the same time, using finite difference in the parallel computing of financial derivatives' price, we can also enjoy a better speed-up ratio.

MapReduce is a simple programming model for developing distributed data intensive applications, and has been used in more and more areas, such as biologic filed [12]. MRPSO [13] utilizes the MapReduce model to parallelize Particle Swarm Optimization, and have good efficient in the detail applications with data intensive and big data set. MRPGA [14] [15], use MapReduce to implement parallel Genetic Algorithms.

In this paper, MapReduce is first applied on option pricing. It takes MRPGA[14], MRPSO[13] as reference, and designs its own model for option pricing with BSDEs on MapReduce. The detailed algorithm and model is given in section 3.

## VI. CONCLUSION

This paper introduces the importance of parallelizing option pricing and MapReduce application. Then it presents the implementation of option pricing with BSDEs on MapReduce, such as mapper and reducer function, the architecture of the model. As possible as we know, it is the first time to use MapReduce in financial filed. Through analyzing of the experiment result, we know that the method implemented on MapReduce is feasible. Although the application of MapReduce in financial computing area is launched, the performance should be improved more. There is lot's of work to be done in the future.

In future, the arithmetic will be optimized, especially the reducer function, to improve the performance of the model on MapReduce. Now only one job is submitted to the master. Later multi-jobs will be submitted to improve the efficiency of the CPUs. We hope multi-dimensions option pricing can be implemented with MapReduce, which needs larger data set and it can get better performance. We hope more work



should be done to push forward the application of MapReduce in cloud computing.

#### ACKNOWLEDGMENT

This work is supported by a grant from science and technology R & D Program of Shandong Province (No. 2009GG10001001).

#### REFERENCES

- [1] El Karoui, N., Peng, S. & Quenez, M.C. (1997b). Backward stochastic differential equations in finance. *Mathematical Finance*, 7, 1–71. El
- [2] Karoui, N.El., Peng, S., Quenez, M.C.: Backward stochastic differential equations in finance. *Mathematical Finance*, Vol. 7, No. 1, (1997)1-71
- [3] Weidong Zhao, Lifeng Chen, Shige Peng, A New Kind of Accurate Numerical Method for Backward Stochastic Differential Equations\_ *SIAM Journal on Scientific Computing*, SISC Volume 28 Issue 4, Pages 1563-1581\_
- [4] Apache.Hadoop.<http://lucene.apache.org/hadoop/>
- [5] Jeffrey Dean and Sanjay Ghemawat, MapReduce: Simplified Data Processing on Large Clusters,OSDI,2004
- [6] <http://patft.uspto.gov/netacgi/nph-Parser?Sect1=PTO1&Sect2=HITOFF&d=PALL&p=1&u=/netahtml/PTO/srchnum.htm&r=1&f=G&l=50&s1=7,650,331.PN.&OS=PN/7,650,331&RS=PN/7,650,331>
- [7] RUPPA K.Thulasiram and Mmitri A.Bondarenko. Performance evaluation of parallel algorithms for pricing multidimensional financial derivatives, 1530-2016/02,2002IEEE
- [8] Kai Huang RUPPA K. Parallel Algorithm for Pricing American Asian Options with Multi-Dimensional Assets.Proceedings of the 19th International Symposium on High Performance Computing Systems and Applications (HPCS'05) 1550-5243/05
- [9] Giorgio Pauletto, Parallel Monte Carlo Methods for Derivative Security Pricing Numerical, NAA '00 Revised Papers from the Second International Conference on Numerical Analysis and Its Applications ©2001 table of contents ISBN:3-540-41814-8
- [10] Jin Suk KIM, Suk Joon BYUN. A Parallel Monte Carlo Simulation on Cluster Systems for Financial Derivatives Pricing 0-7803-9363-5/05 ©2005 IEEE.
- [11] Giray Ökten and Matthew Willyard. Parameterization Based on Randomized Quasi-Monte Carlo Methods, 978-1-4244-1694-3/08©2008 IEEE
- [12] Massimo Gaggero, Simone Leo, Simone Manca, Federico Santoni, Omar Schiaratura, Gianluigi Zanetti, Parallelizing bioinformatics applications with MapReduce, 27 Aug 2008
- [13] Andrew W McNabb, Christopher K. Monson, and Kevin D Seppi, Parallel PSO Using MapReduce, 1-4244-1340-0/07\$25.00 c\_2007 IEEE
- [14] Chao Jin, Christian Vecchiola and Rajkumar Buyya, MRPGA: An Extension of MapReduce for Parallelizing Genetic Algorithms
- [15] Abhishek Verma†, Xavier Llor'a, David E. Goldberg# and Roy H. Campbell, Scaling Genetic Algorithms using MapReduce, 9 Oct 2009
- [16] Sangwon Seo1, Ingook Jang1 HPMR: Prefetching and Pre-shuffling in Shared, Cluster,2009